Bikkad IT

Dream Big! Start Small!! Act Now!!!

Bikkad IT Institute

Pandas DataFrame Notes - Daily Life Edition

Compiled by Bikkad IT Institute | Teaching Notes

Pandas DataFrame Notes

Daily Life Edition - learn DataFrames with food orders and cab rides

Prepared for Bikkad IT Institute Students | Level: Beginner to Intermediate | Environment: Jupyter Notebook or Google Colab

Topic Index

Creating a DataFrame

1. Creating a DataFrame using Lists
2. Creating a DataFrame using Dictionaries
3. Creating a DataFrame using read_csv (food orders)
4. Creating a DataFrame using read_csv (daily rides)

DataFrame Attributes and Methods

5. shape
6. dtypes
7. index
8. columns
9. values
10. head and tail
11. sample
12. info
13. describe
14. isnull
15. duplicated
16. rename

Math Methods

17. sum (axis argument)
18. mean with axis=1
19. var (variance) and std

Selecting Columns from a DataFrame

20. Selecting a single column
21. Selecting multiple columns

Selecting Rows from a DataFrame

22. iloc - single row
23. iloc - multiple rows
24. iloc - fancy indexing

Filtering a DataFrame

30. Find all airport rides
31. How many rides had surge pricing
32. Rides of one type cancelled in one city
33. Rainy day rides with surge (percentage)
34. Late deliveries with a poor rating
35. Biryani orders that got 5 stars
36. Route report between two areas (function)

Adding New Columns

37. Adding a completely new column
38. Adding a column from existing columns

Important DataFrame Functions

39. astype - changing to a smaller number type
40. astype - text to category
41. value_counts - most ordered main dish
42. value_counts - payment mode plot
43. value_counts - how often each area is used
44. sort_values
45. rank
46. sort_index
47. set_index and reset_index
48. unique and nunique
49. isnull, notnull and hasnans
50. dropna
51. fillna
52. drop_duplicates
53. drop
54. apply
55. isin

25. loc - single row
26. loc - slicing with a step
27. loc vs iloc - fancy indexing

Selecting both Rows and Columns

28. iloc with rows and columns
29. loc with rows and columns
56. corr
57. nlargest and nsmallest
58. insert
59. copy
60. groupby
61. to_csv

Datasets: food_orders.csv (1,503 online food-delivery orders - restaurants, dishes, bill, coupons, delivery time and ratings) and daily_rides.csv (1,000 app-cab rides - ride type, distance, fare, surge, weather, payment and ratings), both from the same 5 cities, January - June 2024. Restaurants and people are fictional; the numbers are simulated with realistic daily-life patterns.

About the Datasets

food_orders.csv - one row = one food-delivery order (1,503 rows x 22 columns)

Column	What it holds (in easy words)
order_id	Order number, e.g. FD50000
order_date / day	Date of the order and its weekday
order_hour	Hour the order was placed (0 - 23)
meal_time	Breakfast, Lunch, Snacks, Dinner or Late Night
city / area	Delivery city and area, e.g. Pune - Baner
customer	Who placed the order
restaurant	One of 14 restaurants, e.g. Biryani Junction, Pav Bhaji Point, Dosa Delight
cuisines	Restaurant cuisines joined with |, e.g. Biryani|Hyderabadi|Mughlai
dishes	Dishes in the order joined with |, main (costliest) dish first
veg_nonveg	Veg or Non-Veg restaurant
quantity	Number of plates/items ordered
order_value	Bill amount in rupees before discount
coupon_code	Coupon used, e.g. FEAST20 - empty if none
discount	Rupees saved with the coupon
payment_mode	UPI, Card, Cash or Wallet
delivery_mins	Minutes taken to deliver
customer_rating	Stars (1 - 5) the customer gave - empty if not rated
review	Short written review - empty for most orders
restaurant_rating / restaurant_votes	The restaurant's overall app rating and how many people rated it

daily_rides.csv - one row = one cab ride (1,000 rows x 19 columns)

Column	What it holds (in easy words)
ride_id	Booking number, e.g. RD25000
date / hour	Day and hour (0 - 23) the ride was booked
city	Pune, Mumbai, Bengaluru, Delhi or Hyderabad
pickup_area / drop_area	Where the ride started and ended, e.g. Hinjewadi to Pune Airport
ride_type	Bike, Auto, Mini, Sedan or SUV
rider / driver	Who booked the ride and who drove
distance_km	Trip length in kilometres
duration_mins	Trip time in minutes
weather	Sunny, Cloudy or Rainy
surge	Y if surge (extra) pricing applied, else N

Column	What it holds (in easy words)
`fare`	Amount paid in rupees - empty for cancelled rides
`payment_mode`	UPI, Cash, Card or Wallet
`status`	Completed or Cancelled
`cancelled_by`	Rider or Driver - empty for completed rides
`driver_rating`	Stars (1 - 5) the rider gave the driver - empty if not rated
`tip`	Tip given to the driver in rupees

Creating a DataFrame

1. Creating a DataFrame using Lists

A DataFrame is a table of rows and columns - like an Excel sheet inside Python. From a list of lists, every inner list becomes one row.

Syntax

```
pd.DataFrame(list_of_lists, columns=['col1', 'col2', ...])
```

Example: Four friends' daily routine (hours per day)

```
import numpy as np
import pandas as pd

routine_data = [
    [7, 5, 3],
    [6, 3, 6],
    [8, 6, 2],
    [5, 2, 7]
]
print("List of lists:", routine_data)

df = pd.DataFrame(routine_data,
                  columns=['sleep_hrs', 'study_hrs', 'phone_hrs'])
print(df)
```

Output / Result

```
List of lists: [[7, 5, 3], [6, 3, 6], [8, 6, 2], [5, 2, 7]]
   sleep_hrs  study_hrs  phone_hrs
0          7          5          3
1          6          3          6
2          8          6          2
3          5          2          7
```

Explanation

Each inner list is one friend (one row): hours of sleep, study and phone use. columns names the three columns, and pandas adds the default row labels 0 to 3.

2. Creating a DataFrame using Dictionaries

With a dictionary, every key becomes a column name and its list becomes that column. set_index() then makes one column the row labels.

Syntax

```
pd.DataFrame(dictionary)
df.set_index('column', inplace=True)
```

Example: A class routine survey with names as the index

```
routine_dict = {
    'name': ['aarav', 'diya', 'kabir', 'meera', 'rohan', 'sana'],
    'sleep_hrs': [7, 6, 8, 5, 0, 0],
    'study_hrs': [5, 3, 6, 2, 0, 0],
    'phone_hrs': [3, 6, 2, 7, 0, 0]
}

routine = pd.DataFrame(routine_dict)
print(routine)

routine.set_index('name', inplace=True)
print(routine)
```

Output / Result

```
    name  sleep_hrs  study_hrs  phone_hrs
0  aarav          7          5          3
1   diya          6          3          6
2  kabir          8          6          2
3  meera          5          2          7
4  rohan          0          0          0
5   sana          0          0          0
       sleep_hrs  study_hrs  phone_hrs
name
aarav          7          5          3
diya           6          3          6
kabir          8          6          2
meera          5          2          7
rohan          0          0          0
sana           0          0          0
```

Explanation

The first print shows name as a normal column. After set_index('name', inplace=True) the names become the row labels. rohan and sana have not filled the survey yet, so all their values are 0.

3. Creating a DataFrame using read_csv (food orders)

pd.read_csv() reads a CSV file and returns a DataFrame. The first line of the file becomes the column names.

Syntax

```
pd.read_csv('file.csv')
```

Example: Loading the food-delivery orders

```
food = pd.read_csv('food_orders.csv')
print(type(food))
print(food)
```

Output / Result

```
<class 'pandas.DataFrame'>
     order_id  order_date  ... restaurant_rating  restaurant_votes
0     FD50000  2024-06-30  ...               3.8              2100
1     FD49999  2024-06-30  ...               4.4             18400
2     FD49998  2024-06-30  ...               4.4              6100
3     FD49997  2024-06-30  ...               4.4             18400
4     FD49996  2024-06-30  ...               4.4              6100
...       ...         ...  ...               ...               ...
1498  FD48505  2024-01-01  ...               4.4             18400
1499  FD48504  2024-01-01  ...               4.0             16800
1500  FD48503  2024-01-01  ...               4.0              6400
1501  FD48502  2024-01-01  ...               4.4             18400
1502  FD48501  2024-01-01  ...               4.1              3900

[1503 rows x 22 columns]
```

Explanation

1,503 food-delivery orders placed between January and June 2024 in Pune, Mumbai, Bengaluru, Delhi and Hyderabad, with 22 columns. The table is too wide to show, so print() shows the first and last 5 rows, hides the middle columns with '...', and reports the full size at the bottom.

4. Creating a DataFrame using read_csv (daily rides)

The same function reads any CSV file - here, 1,000 app-cab rides booked in five cities.

Syntax

```
pd.read_csv('file.csv')
```

Example: Loading the daily rides dataset

```
rides = pd.read_csv('daily_rides.csv')
print(type(rides))
print(rides)
```

Output / Result

```
<class 'pandas.DataFrame'>
     ride_id        date  hour  ... cancelled_by driver_rating tip
0    RD25000  2024-06-30    18  ...          NaN           4.0  10
1    RD24999  2024-06-30    15  ...          NaN           4.0  50
2    RD24998  2024-06-30    12  ...          NaN           5.0   0
3    RD24997  2024-06-30     9  ...          NaN           NaN  10
4    RD24996  2024-06-30     9  ...          NaN           4.0  20
..       ...         ...   ...  ...          ...           ...  ..
995  RD24005  2024-01-02     2  ...          NaN           4.0  20
996  RD24004  2024-01-02     0  ...          NaN           4.0   0
997  RD24003  2024-01-01    17  ...          NaN           4.0  10
998  RD24002  2024-01-01    15  ...          NaN           NaN  30
999  RD24001  2024-01-01    14  ...          NaN           5.0   0

[1000 rows x 19 columns]
```

Explanation

1,000 cab rides (Bike, Auto, Mini, Sedan and SUV) booked between January and June 2024 in Pune, Mumbai, Bengaluru, Delhi and Hyderabad, with 19 columns.

DataFrame Attributes and Methods

5. shape

shape tells you the size of a DataFrame as (rows, columns).

Syntax

```
df.shape
```

Example: Size of all three DataFrames

```
print("food   :", food.shape)
print("rides  :", rides.shape)
print("routine:", routine.shape)
```

Output / Result

```
food   : (1503, 22)
rides  : (1000, 19)
routine: (6, 3)
```

Explanation

food has 1,503 rows and 22 columns, rides 1,000 rows and 19 columns, and routine 6 rows and 3 columns (name is now the index, so it is not counted as a column).

6. dtypes

dtypes shows the data type of every column - numbers (int64, float64) or text (str).

Syntax

```
df.dtypes
```

Example: Data type of every ride column

```
print(rides.dtypes)
```

Output / Result

```
ride_id              str
date                 str
hour               int64
city                 str
pickup_area          str
drop_area            str
ride_type            str
rider                str
driver               str
distance_km        float64
duration_mins        int64
weather              str
surge                str
fare               float64
payment_mode         str
status               str
cancelled_by         str
driver_rating      float64
tip                  int64
dtype: object
```

Explanation

hour, duration_mins and tip are whole numbers (int64); distance_km, fare and driver_rating are decimals (float64). fare and driver_rating are float because cancelled rides have no fare and many riders skip the rating - a column with empty cells becomes float.

7. index

index holds the row labels of a DataFrame.

Syntax

```
df.index
```

Example: Row labels of three DataFrames

```
print(food.index)
print(rides.index)
print(routine.index)
```

Output / Result

```
RangeIndex(start=0, stop=1503, step=1)
RangeIndex(start=0, stop=1000, step=1)
Index(['aarav', 'diya', 'kabir', 'meera', 'rohan', 'sana'], dtype='str', name='name')
```

Explanation

food and rides use the default RangeIndex (0, 1, 2 ...). routine uses the names we set with set_index().

8. columns

columns lists all column names - useful when you forget the exact spelling of a column.

Syntax

```
df.columns
```

Example: Column names of the food orders and routine tables

```
print(food.columns)
print(routine.columns)
```

Output / Result

```
Index(['order_id', 'order_date', 'day', 'order_hour', 'meal_time', 'city',
       'area', 'customer', 'restaurant', 'cuisines', 'dishes', 'veg_nonveg',
       'quantity', 'order_value', 'coupon_code', 'discount', 'payment_mode',
       'delivery_mins', 'customer_rating', 'review', 'restaurant_rating',
       'restaurant_votes'],
      dtype='str')
Index(['sleep_hrs', 'study_hrs', 'phone_hrs'], dtype='str')
```

Explanation

food has 22 column names. routine has only sleep_hrs, study_hrs and phone_hrs, because name moved to the index.

9. values

values returns only the data (no labels) as a two-dimensional NumPy array.

Syntax

```
df.values
```

Example: Raw data of the routine table

```
routine_values = routine.values
print(routine_values)
print(type(routine_values))
```

Output / Result

```
[[7 5 3]
 [6 3 6]
 [8 6 2]
 [5 2 7]
 [0 0 0]
 [0 0 0]]
<class 'numpy.ndarray'>
```

Explanation

Each inner array is one person's sleep, study and phone hours. The names are gone because values drops all labels.

10. head and tail

head(n) shows the first n rows and tail(n) the last n rows (5 by default).

Syntax

```
df.head(n)
df.tail(n)
```

Example: Newest food orders and oldest rides

```
first_two = food.head(2)
print(first_two)

last_two = rides.tail(2)
print(last_two)
```

Output / Result

```
  order_id  order_date  ... restaurant_rating  restaurant_votes
0  FD50000  2024-06-30  ...               3.8              2100
1  FD49999  2024-06-30  ...               4.4             18400

[2 rows x 22 columns]
     ride_id        date  hour  ... cancelled_by driver_rating tip
998  RD24002  2024-01-01    15  ...          NaN           NaN  30
999  RD24001  2024-01-01    14  ...          NaN           5.0   0

[2 rows x 19 columns]
```

Explanation

Both files are sorted newest-first, so head(2) shows the last orders of 30 June and tail(2) of rides shows the very first rides of 1 January 2024.

11. sample

sample(n) picks n random rows - a fair way to peek into the middle of a big dataset.

Syntax

```
df.sample(n, random_state=number)
```

Example: Five random rides

```
random_rides = rides.sample(5, random_state=7)
print(random_rides[['city', 'ride_type', 'distance_km', 'fare']])
```

Output / Result

```
          city ride_type  distance_km   fare
778  Hyderabad      Mini          4.9  124.0
334     Mumbai     Sedan         17.1  378.0
271       Pune       SUV          3.2  177.0
802     Mumbai      Mini          3.4  101.0
216       Pune     Sedan         18.9  410.0
```

Explanation

random_state fixes the randomness, so everyone gets the same 5 rows. (Selecting a few columns keeps the output readable - column selection is explained later.)

12. info

info() prints the number of rows, each column's name, its non-null (filled) count, data type and memory.

Syntax

```
df.info()
```

Example: Summary of the rides dataset

```
rides.info()
```

Output / Result

```
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 19 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   ride_id        1000 non-null   str
 1   date           1000 non-null   str
 2   hour           1000 non-null   int64
 3   city           1000 non-null   str
 4   pickup_area    1000 non-null   str
 5   drop_area      1000 non-null   str
 6   ride_type      1000 non-null   str
 7   rider          1000 non-null   str
 8   driver         1000 non-null   str
 9   distance_km    1000 non-null   float64
 10  duration_mins  1000 non-null   int64
 11  weather        1000 non-null   str
 12  surge          1000 non-null   str
 13  fare           903 non-null    float64
 14  payment_mode   1000 non-null   str
 15  status         1000 non-null   str
 16  cancelled_by   97 non-null     str
 17  driver_rating  795 non-null    float64
 18  tip            1000 non-null   int64
dtypes: float64(3), int64(3), str(13)
memory usage: 148.6 KB
```

Explanation

Compare each Non-Null Count with 1,000: fare is missing for the 97 cancelled rides, driver_rating for 205 rides (cancelled rides plus riders who skipped rating), and cancelled_by is filled only for the 97 cancelled rides.

13. describe

describe() gives count, mean, standard deviation, minimum, quartiles and maximum of every numeric column.

Syntax

```
df.describe()
```

Example: Statistics of the food orders

```
numbers = food[['quantity', 'order_value', 'discount', 'delivery_mins',
                'customer_rating']]
stats = numbers.describe()
print(stats.round(1))
```

Output / Result

```
       quantity  order_value  discount  delivery_mins  customer_rating
count    1503.0       1503.0    1503.0         1503.0           1280.0
mean        3.2        531.6      27.7           33.7              4.3
std         1.4        384.9      51.9            7.8              0.7
min         1.0         25.0       0.0           12.0              2.0
25%         2.0        249.5       0.0           28.0              4.0
50%         3.0        450.0       0.0           34.0              4.0
75%         4.0        740.0      24.0           39.0              5.0
max         6.0       1930.0     150.0           61.0              5.0
```

Explanation

An average order has about 3 items, costs about Rs. 530 and arrives in about 34 minutes; customers give about 4.3 stars on average. customer_rating has a lower count because some customers skip rating.

14. isnull

isnull() marks missing values as True; adding .sum() counts the missing values in each column.

Syntax

```
df.isnull().sum()
```

Example: Missing values in every ride column

```
missing = rides.isnull().sum()
print(missing)
```

Output / Result

```
ride_id            0
date               0
hour               0
city               0
pickup_area        0
drop_area          0
ride_type          0
rider              0
driver             0
distance_km        0
duration_mins      0
weather            0
surge              0
fare              97
payment_mode       0
status             0
cancelled_by     903
driver_rating    205
tip                0
dtype: int64
```

Explanation

fare (97) and cancelled_by (903) are two sides of the same story: a cancelled ride has no fare, and a completed ride has nobody who cancelled it. 205 rides have no driver rating.

15. duplicated

duplicated() returns True for a row that is an exact copy of an earlier row; .sum() counts them.

Syntax

```
df.duplicated().sum()
```

Example: Double-placed orders and duplicate survey rows

```
print("Duplicate food orders:", food.duplicated().sum())

print(routine.duplicated())
print("Duplicate routine rows:", routine.duplicated().sum())
```

Output / Result

```
Duplicate food orders: 3
name
aarav    False
diya     False
kabir    False
meera    False
rohan    False
sana      True
dtype: bool
Duplicate routine rows: 1
```

Explanation

3 orders were placed twice because of an app glitch. In routine, sana is marked as a duplicate because her values (0, 0, 0) are the same as rohan's - duplicated() compares only values, not index labels.

16. rename

rename() changes column names using a dictionary {old_name: new_name}.

Syntax

```
df.rename(columns={'old': 'new'}, inplace=True)
```

Example: Shorter column names

```
print(routine.columns)

routine.rename(columns={'sleep_hrs': 'sleep', 'study_hrs': 'study',
                        'phone_hrs': 'phone'}, inplace=True)
print(routine)
```

Output / Result

```
Index(['sleep_hrs', 'study_hrs', 'phone_hrs'], dtype='str')
       sleep  study  phone
name
aarav      7      5      3
diya       6      3      6
kabir      8      6      2
meera      5      2      7
rohan      0      0      0
sana       0      0      0
```

Explanation

inplace=True changes routine itself, so from now on its columns are sleep, study and phone.

Math Methods

17. sum (axis argument)

Math methods work on every column at once. axis=0 (default) goes down the columns; axis=1 goes across the rows.

Syntax

```
df.sum(axis=0)
df.sum(axis=1)
```

Example: Class totals and each person's total hours

```
column_totals = routine.sum(axis=0)
print(column_totals)

row_totals = routine.sum(axis=1)
print(row_totals)
```

Output / Result

```
sleep    26
study    16
phone    18
dtype: int64
name
aarav    15
diya     15
kabir    16
meera    14
rohan     0
sana      0
dtype: int64
```

Explanation

axis=0 adds down each column (the whole group sleeps 26 hours in total); axis=1 adds across each row (aarav spends 15 hours a day on these three activities).

18. mean with axis=1

With axis=1 the calculation happens across each row instead of down each column.

Syntax

```
df.mean(axis=1)
```

Example: Average of each person's three activities

```
row_means = routine.mean(axis=1)
print(row_means.round(2))
```

Output / Result

```
name
aarav    5.00
diya     5.00
kabir    5.33
meera    4.67
rohan    0.00
sana     0.00
dtype: float64
```

Explanation

Each person's sleep, study and phone hours are averaged. This mainly shows how axis=1 works across a row.

19. var (variance) and std

var() and std() measure how spread out the values in each column are.

Syntax

```
df.var()
df.std()
```

Example: How different are the students' habits?

```
variance = routine.var()
print(variance.round(2))

std_dev = routine.std()
print(std_dev.round(2))
```

Output / Result

```
sleep    12.27
study     6.27
phone     8.80
dtype: float64
sleep    3.50
study    2.50
phone    2.97
dtype: float64
```

Explanation

A bigger number means the values are more spread out. Sleep varies the most here because rohan and sana have 0; standard deviation is the square root of variance, in hours.

Selecting Columns from a DataFrame

20. Selecting a single column

df['column'] returns one column as a Series.

Syntax

```
df['col']
```

Example: Order values and ride fares

```
bills = food['order_value']
print(type(bills))
print(bills.head())

fares = rides['fare']
print(fares.head(3))
```

Output / Result

```
<class 'pandas.Series'>
0     730
1    1140
2     280
3    1660
4     160
Name: order_value, dtype: int64
0    360.0
1    509.0
2    167.0
Name: fare, dtype: float64
```

Explanation

A single column comes back as a Series with the same row labels as the DataFrame. order_value is a whole number, while ride fares are decimals because cancelled rides have no fare.

21. Selecting multiple columns

Put a list of column names inside the brackets - df[[...]] - to get a smaller DataFrame.

Syntax

```
df[['col1', 'col2', 'col3']]
```

Example: Who ordered what, and ride routes

```
order_view = food[['customer', 'restaurant', 'dishes']]
print(order_view.head(3))

route_view = rides[['pickup_area', 'drop_area', 'ride_type']]
print(route_view.head(3))
```

Output / Result

```
    customer        restaurant                                        dishes
0   Rahul S.  The Healthy Bowl      Protein Bowl|Quinoa Salad|Fruit Smoothie
1    Riya K.  Biryani Junction  Chicken Biryani|Veg Biryani|Double Ka Meetha
2  Pranav A.  Paratha Paradise             Paneer Paratha|Aloo Paratha|Lassi
      pickup_area  drop_area ride_type
0          Bandra        CST      Mini
1  Mumbai Airport     Bandra      Mini
2        Hadapsar  Hinjewadi      Bike
```

Explanation

The outer brackets select, the inner brackets are a Python list of names. Columns appear in the order you list them.

Selecting Rows from a DataFrame

22. iloc - single row

iloc selects rows by their integer position (0 = first row).

Syntax

```
df.iloc[position]
```

Example: The 6th ride

```
sixth_ride = rides.iloc[5]
print(sixth_ride)
```

Output / Result

```
ride_id                RD24995
date                2024-06-30
hour                         6
city                    Mumbai
pickup_area              Powai
drop_area               Andheri
ride_type                  Mini
rider                Pranav A.
driver           Prakash Singh
distance_km                5.7
duration_mins               21
weather                  Sunny
surge                        N
fare                       NaN
payment_mode               UPI
status               Cancelled
cancelled_by             Rider
driver_rating              NaN
tip                          0
Name: 5, dtype: object
```

Explanation

One row comes back as a Series: the column names become its labels. This 6th ride was cancelled by the rider, so fare and driver_rating are NaN.

23. iloc - multiple rows

A slice of positions returns several rows. The end position is not included.

Syntax

```
df.iloc[start:end]
```

Example: The first five food orders

```
first_five = food.iloc[:5]
print(first_five[['restaurant', 'order_value', 'delivery_mins']])
```

Output / Result

```
          restaurant  order_value  delivery_mins
0  The Healthy Bowl          730             35
1  Biryani Junction         1140             36
2  Paratha Paradise          280             49
3  Biryani Junction         1660             34
4  Paratha Paradise          160             25
```

Explanation

iloc[:5] gives positions 0, 1, 2, 3 and 4 - five rows.

24. iloc - fancy indexing

A list of positions returns exactly those rows, in that order.

Syntax

```
df.iloc[[p1, p2, p3]]
```

Example: Rides at positions 0, 4 and 5

```
chosen = rides.iloc[[0, 4, 5]]
print(chosen[['city', 'pickup_area', 'drop_area', 'fare']])
```

Output / Result

```
     city pickup_area       drop_area   fare
0  Mumbai      Bandra             CST  360.0
4  Mumbai       Dadar  Mumbai Airport  278.0
5  Mumbai       Powai         Andheri    NaN
```

Explanation

Only rows 0, 4 and 5 are returned - the rows in between are skipped.

25. loc - single row

loc selects rows by their index label.

Syntax

```
df.loc['label']
```

Example: One person's routine by name

```
print(routine)

kabir = routine.loc['kabir']
print(kabir)
```

Output / Result

```
       sleep  study  phone
name
aarav      7      5      3
diya       6      3      6
kabir      8      6      2
meera      5      2      7
rohan      0      0      0
sana       0      0      0
sleep    8
study    6
phone    2
Name: kabir, dtype: int64
```

Explanation

Because names are the index of routine, we can fetch a person directly by name: kabir sleeps 8 hours, studies 6 and uses the phone only 2.

26. loc - slicing with a step

Label slices include the end label. A third number is the step.

Syntax

```
df.loc['start':'end':step]
```

Example: Every second person from aarav to rohan

```
every_second = routine.loc['aarav':'rohan':2]
print(every_second)
```

Output / Result

```
       sleep  study  phone
name
aarav      7      5      3
kabir      8      6      2
rohan      0      0      0
```

Explanation

The slice starts at aarav, ends at rohan (included) and the step 2 picks aarav, kabir and rohan.

27. loc vs iloc - fancy indexing

Both accept a list: loc a list of labels, iloc a list of positions.

Syntax

```
df.loc[['label1', 'label2']]
df.iloc[[0, 3, 4]]
```

Example: Choosing people by name and by position

```
by_label = routine.loc[['aarav', 'sana', 'diya']]
print(by_label)

by_position = routine.iloc[[0, 3, 4]]
print(by_position)
```

Output / Result

```
       sleep  study  phone
name
aarav      7      5      3
sana       0      0      0
diya       6      3      6
       sleep  study  phone
name
aarav      7      5      3
meera      5      2      7
rohan      0      0      0
```

Explanation

loc uses the names we see; iloc counts positions from 0 (0 = aarav, 3 = meera, 4 = rohan).

Selecting both Rows and Columns

28. iloc with rows and columns

iloc[rows, columns] cuts out a rectangle using positions.

Syntax

```
df.iloc[row_slice, col_slice]
```

Example: A small block of food orders by position

```
block = food.iloc[0:3, 4:9]
print(block)
```

Output / Result

```
  meal_time    city        area   customer        restaurant
0    Dinner  Mumbai       Thane   Rahul S.  The Healthy Bowl
1    Dinner  Mumbai     Andheri    Riya K.  Biryani Junction
2    Dinner   Delhi  Karol Bagh  Pranav A.  Paratha Paradise
```

Explanation

Rows 0 to 2 and column positions 4 to 8 (meal_time, city, area, customer, restaurant). The end positions 3 and 9 are excluded.

29. loc with rows and columns

loc[rows, columns] does the same using labels - both ends included.

Syntax

```
df.loc[row_labels, 'col_start':'col_end']
```

Example: Rows 0 to 2, columns ride_type to distance_km

```
block = rides.loc[0:2, 'ride_type':'distance_km']
print(block)
```

Output / Result

```
  ride_type      rider         driver  distance_km
0      Mini  Nikhil S.    Vijay Naik         20.7
1      Mini   Aarav K.    Vijay Naik         30.6
2      Bike    Riya S.  Rajesh Kumar         12.6
```

Explanation

With loc, the row labels 0 to 2 and the columns ride_type to distance_km are all included.

Filtering a DataFrame

30. Find all airport rides

A condition creates a True/False mask; df[mask] keeps only the True rows.

Syntax

```
mask = df['col'].str.contains('text')
df[mask]
```

Example: Rides to or from the airport

```
to_airport = rides['drop_area'].str.contains('Airport')
from_airport = rides['pickup_area'].str.contains('Airport')

airport_rides = rides[to_airport | from_airport]
print(airport_rides[['city', 'pickup_area', 'drop_area',
                     'distance_km']].head())
print("Airport rides:", airport_rides.shape[0])
```

Output / Result

```
         city        pickup_area       drop_area  distance_km
1      Mumbai     Mumbai Airport          Bandra         30.6
4      Mumbai              Dadar  Mumbai Airport         20.7
12     Mumbai     Mumbai Airport          Bandra         15.4
19       Pune              Baner    Pune Airport         26.2
22  Bengaluru  Bengaluru Airport         MG Road         20.0
Airport rides: 263
```

Explanation

Two masks are joined with | (or): a ride is kept if it goes TO the airport or comes FROM the airport. 263 of the 1,000 rides are airport rides - and they are long (15 to 40 km).

31. How many rides had surge pricing

shape[0] of a filtered DataFrame counts how many rows matched the condition.

Syntax

```
df[df['col'] == value].shape[0]
```

Example: Counting surge rides

```
surge_rides = rides[rides['surge'] == 'Y']
print(surge_rides[['city', 'hour', 'weather', 'fare']].head())

print("Surge rides:", surge_rides.shape[0])
```

Output / Result

```
         city  hour weather   fare
2        Pune    12   Rainy  167.0
3   Hyderabad     9   Rainy  144.0
8       Delhi    21   Rainy  136.0
9       Delhi    17   Rainy   97.0
12     Mumbai     9   Sunny  563.0
Surge rides: 322
```

Explanation

322 rides (about one in three) had surge pricing. Surge shows up more in rainy weather and in office hours.

32. Rides of one type cancelled in one city

Combine conditions with & (and). Each condition must be inside its own brackets.

Syntax

```
df[(cond1) & (cond2)]
```

Example: Cancelled auto rides in Pune

```
in_pune = rides['city'] == 'Pune'
is_auto = rides['ride_type'] == 'Auto'
cancelled = rides['status'] == 'Cancelled'

pune_auto_cancel = rides[in_pune & is_auto & cancelled]
print(pune_auto_cancel[['date', 'pickup_area', 'drop_area',
                        'cancelled_by']])
print("Cancelled autos in Pune:", pune_auto_cancel.shape[0])
```

Output / Result

```
           date   pickup_area     drop_area cancelled_by
35   2024-06-25  Pune Airport      Hadapsar       Driver
413  2024-04-16         Baner  Shivajinagar        Rider
562  2024-03-21  Pune Airport     Hinjewadi       Driver
Cancelled autos in Pune: 3
```

Explanation

All three conditions must be True. Only 3 auto rides in Pune were cancelled - 2 by the driver, 1 by the rider.

33. Rainy day rides with surge (percentage)

A filtered count divided by the total gives a percentage.

Syntax

```
df[cond1 & cond2].shape[0] / df[cond1].shape[0] * 100
```

Example: Does rain make rides costlier?

```
rainy = rides[rides['weather'] == 'Rainy']
print("Rainy rides:", rainy.shape[0])

rainy_surge = rainy[rainy['surge'] == 'Y']
percent = rainy_surge.shape[0] / rainy.shape[0] * 100
print("Rainy rides with surge:", round(percent, 2), "%")
```

Output / Result

```
Rainy rides: 149
Rainy rides with surge: 61.74 %
```

Explanation

149 rides were booked in the rain and 61.74% of them had surge pricing - roughly double the surge rate on sunny days. Rain really does make rides costlier!

34. Late deliveries with a poor rating

Numeric conditions can be combined the same way.

Syntax

```
df[(df['col1'] > x) & (df['col2'] <= y)]
```

Example: Delivered after 45 minutes AND rated 2 stars or less

```
late = food['delivery_mins'] > 45
poor_rating = food['customer_rating'] <= 2

angry_orders = food[late & poor_rating]
print(angry_orders[['restaurant', 'delivery_mins', 'customer_rating',
                    'review']].head())
print("Such orders:", angry_orders.shape[0])
```

Output / Result

```
            restaurant  delivery_mins  customer_rating                   review
315   Biryani Junction             56              2.0  Delivery took too long.
504        Burger Barn             49              2.0  Delivery took too long.
788     Pav Bhaji Point             50              2.0                      NaN
1280       Cake Corner             61              2.0                      NaN
1291       Burger Barn             48              2.0                      NaN
Such orders: 6
```

Explanation

Both conditions must be True: the food took more than 45 minutes AND the customer gave 2 stars or less. Only 6 orders are this bad - and their reviews tell the story.

35. Biryani orders that got 5 stars

str.contains() checks whether a text column contains a word - perfect for dish lists joined with |.

Syntax

```
df['col'].str.contains('word')
```

Example: Orders with any biryani, rated 5 stars

```
mask1 = food['dishes'].str.contains('Biryani')
mask2 = food['customer_rating'] == 5

happy_biryani = food[mask1 & mask2]
print(happy_biryani[['city', 'dishes', 'customer_rating']].head())
print("Count:", happy_biryani.shape[0])
```

Output / Result

```
         city                                          dishes  customer_rating
1      Mumbai     Chicken Biryani|Veg Biryani|Double Ka Meetha              5.0
11       Pune      Mutton Biryani|Veg Biryani|Double Ka Meetha              5.0
13       Pune                Chicken Biryani|Veg Biryani|Raita              5.0
42  Hyderabad  Mutton Biryani|Chicken Biryani|Double Ka Meetha              5.0
51       Pune                                     Veg Biryani              5.0
Count: 70
```

Explanation

str.contains('Biryani') is True for Chicken, Mutton or Veg Biryani anywhere in the dish list. 70 biryani orders got a perfect 5-star rating.

36. Route report between two areas (function)

Putting a filter inside a function makes it reusable for any two places.

Syntax

```
def route_report(area1, area2): ...
```

Example: All rides between two areas, in both directions

```
def route_report(area1, area2):
    a_to_b = ((rides['pickup_area'] == area1) &
              (rides['drop_area'] == area2))
    b_to_a = ((rides['pickup_area'] == area2) &
              (rides['drop_area'] == area1))
    trips = rides[a_to_b | b_to_a]
    print("Rides between", area1, "and", area2, ":", trips.shape[0])
    return trips[['pickup_area', 'drop_area', 'ride_type', 'fare']]

print(route_report('Hinjewadi', 'Pune Airport'))
```

Output / Result

```
Rides between Hinjewadi and Pune Airport : 8
      pickup_area     drop_area ride_type    fare
82   Pune Airport     Hinjewadi      Mini   364.0
91   Pune Airport     Hinjewadi      Mini   362.0
464     Hinjewadi  Pune Airport      Mini   394.0
474  Pune Airport     Hinjewadi      Auto   410.0
554  Pune Airport     Hinjewadi      Bike     NaN
562  Pune Airport     Hinjewadi      Auto     NaN
742  Pune Airport     Hinjewadi      Auto   310.0
975     Hinjewadi  Pune Airport       SUV  1337.0
```

Explanation

A trip can go either way, so both directions are joined with | (or). 8 rides connected Hinjewadi and Pune Airport; the SUV cost Rs. 1,337, while the cancelled rides have no fare.

Adding New Columns

37. Adding a completely new column

Assigning a single value to a new column name fills that value in every row.

Syntax

```
df['new_col'] = value
```

Example: Country column for every ride

```
rides['country'] = 'India'
print(rides[['ride_id', 'city', 'country']].head(3))
print(rides.shape)
```

Output / Result

```
   ride_id    city country
0  RD25000  Mumbai   India
1  RD24999  Mumbai   India
2  RD24998    Pune   India
(1000, 20)
```

Explanation

country is added as the 20th column with 'India' in every row.

38. Adding a column from existing columns

A new column can be built from an existing one - here the first dish in the list, which is the main (costliest) dish of the order.

Syntax

```
df['new'] = df['col'].str.split('|').apply(lambda x: x[0])
```

Example: Main dish of every order

```
dish_lists = food['dishes'].str.split('|')
print(dish_lists.head(2))

food['main_dish'] = dish_lists.apply(lambda x: x[0])
print(food[['dishes', 'main_dish']].head())
```

Output / Result

```
0        [Protein Bowl, Quinoa Salad, Fruit Smoothie]
1    [Chicken Biryani, Veg Biryani, Double Ka Meetha]
Name: dishes, dtype: object
                                              dishes        main_dish
0           Protein Bowl|Quinoa Salad|Fruit Smoothie     Protein Bowl
1       Chicken Biryani|Veg Biryani|Double Ka Meetha  Chicken Biryani
2                   Paneer Paratha|Aloo Paratha|Lassi   Paneer Paratha
3  Mutton Biryani|Chicken Biryani|Veg Biryani|Dou...   Mutton Biryani
4                                Aloo Paratha|Lassi     Aloo Paratha
```

Explanation

Dishes are listed from the costliest to the cheapest, so the first one is the main dish. split('|') turns the text into a list and apply() takes its first item.

Important DataFrame Functions

39. astype - changing to a smaller number type

astype() converts a column to another data type. A smaller type saves memory.

Syntax

```
df['col'] = df['col'].astype('int16')
```

Example: Quantity from int64 to int16

```
print("Before:", food['quantity'].dtype,
      food['quantity'].memory_usage(), "bytes")

food['quantity'] = food['quantity'].astype('int16')
print("After :", food['quantity'].dtype,
      food['quantity'].memory_usage(), "bytes")
```

Output / Result

```
Before: int64 12156 bytes
After : int16 3138 bytes
```

Explanation

Nobody orders thousands of items at once, so int16 (2 bytes per value) is enough instead of int64 (8 bytes) - memory drops from 12,156 to 3,138 bytes.

40. astype - text to category

Columns with a few repeating text values (like city names) can be stored as category.

Syntax

```
df['col'] = df['col'].astype('category')
```

Example: City and ride type as category

```
before = rides[['city', 'ride_type']].memory_usage(deep=True).sum()
print("Before (bytes):", before)

rides['city'] = rides['city'].astype('category')
rides['ride_type'] = rides['ride_type'].astype('category')

after = rides[['city', 'ride_type']].memory_usage(deep=True).sum()
print("After  (bytes):", after)
```

Output / Result

```
Before (bytes): 124357
After  (bytes): 2755
```

Explanation

Only 5 cities and 5 ride types repeat across 1,000 rows. category stores each name once plus a tiny code per row, cutting memory by about 98%.

41. value_counts - most ordered main dish

value_counts() counts how many times each value appears, largest first.

Syntax

```
df['col'].value_counts()
```

Example: Which dish do people order the most?

```
top_dishes = food['main_dish'].value_counts()
print(top_dishes.head(8))
```

Output / Result

```
main_dish
Pav Bhaji                205
Masala Dosa              113
Mutton Biryani            97
Butter Chicken            87
Chicken Burger            85
Chicken Biryani           81
Chocolate Truffle Cake    74
Farmhouse Pizza           55
Name: count, dtype: int64
```

Explanation

Pav Bhaji is the most ordered main dish (205 orders), followed by Masala Dosa and Mutton Biryani - every city has its favourite!

42. value_counts - payment mode plot

The result of value_counts() can be plotted directly.

Syntax

```
df['col'].value_counts().plot(kind='pie')
```

Example: How do riders pay?

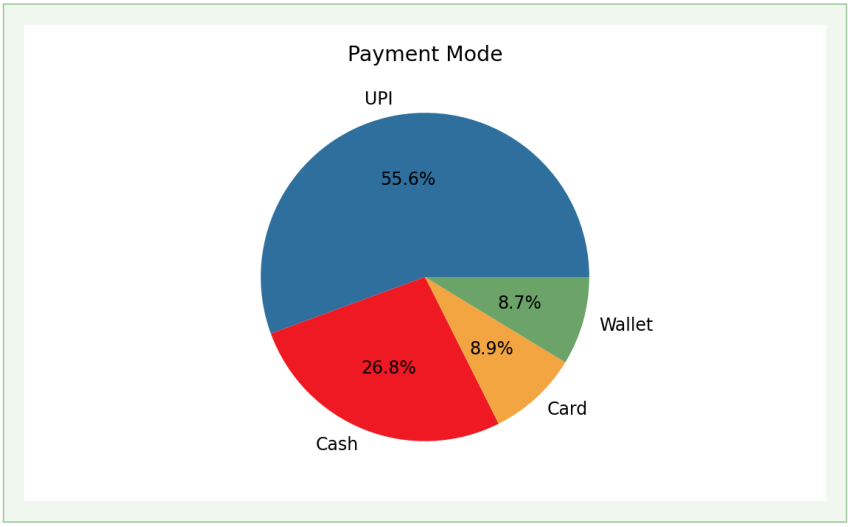
```
import matplotlib.pyplot as plt

payments = rides['payment_mode'].value_counts()
print(payments)

payments.plot(kind='pie', autopct='%1.1f%%',
              colors=['#2E6F9E', '#EF1923', '#F2A541', '#6BA368'])
plt.title('Payment Mode')
plt.ylabel('')
plt.show()
```

Output / Result

```
payment_mode
UPI       556
Cash      268
Card       89
Wallet     87
Name: count, dtype: int64
```



Explanation

More than half the riders pay by UPI; cash is still second. autopct writes the percentage on each slice.

43. value_counts - how often each area is used

Counting two columns and adding the results gives a combined total per area.

Syntax

```
df['col1'].value_counts() + df['col2'].value_counts()
```

Example: Busiest pickup + drop areas

```
pickups = rides['pickup_area'].value_counts()
drops = rides['drop_area'].value_counts()

total = pickups + drops
print(total.sort_values(ascending=False).head(8))
```

Output / Result

```
pickup_area
Baner           90
Shivajinagar    79
Hadapsar        78
Powai           77
Pune Station    77
Hinjewadi       73
Thane           72
Kothrud         71
Name: count, dtype: int64
```

Explanation

An area can be a pickup or a drop, so both counts are added. Baner in Pune is the busiest area in the data.

44. sort_values

sort_values() orders rows by one or more columns. Key arguments: ascending, na_position, inplace.

Syntax

```
df.sort_values(['col1', 'col2'], ascending=[True, False],
    na_position='first')
```

Example: Costliest rides, and sorting by two columns

```
view = rides[['city', 'ride_type', 'distance_km', 'fare']]

costliest = view.sort_values('fare', ascending=False)
print(costliest.head())

by_city_fare = view.sort_values(['city', 'fare'],
                                ascending=[True, False],
                                na_position='last')
print(by_city_fare.head())
```

Output / Result

```
          city ride_type  distance_km    fare
975       Pune       SUV         28.8  1337.0
923  Hyderabad       SUV         26.6  1291.0
652     Mumbai       SUV         28.7  1236.0
86      Mumbai     Sedan         29.3  1112.0
784       Pune       SUV         22.8  1102.0
          city ride_type  distance_km   fare
138  Bengaluru     Sedan         24.5  948.0
860  Bengaluru       SUV         18.4  903.0
756  Bengaluru       SUV         29.6  810.0
108  Bengaluru     Sedan         39.0  772.0
956  Bengaluru      Auto         37.3  698.0
```

Explanation

The first sort puts the costliest rides on top - long SUV and Sedan trips. The second sorts by city (A-Z) and, within each city, by fare from high to low; the NaN fares of cancelled rides go last.

45. rank

rank() gives each value its position (1 = first).

Syntax

```
df['col'].rank(ascending=False)
```

Example: Who sleeps the most?

```
routine['sleep_rank'] = routine['sleep'].rank(ascending=False)
print(routine.sort_values('sleep_rank'))
```

Output / Result

```
       sleep  study  phone  sleep_rank
name
kabir      8      6      2         1.0
aarav      7      5      3         2.0
diya       6      3      6         3.0
meera      5      2      7         4.0
rohan      0      0      0         5.5
sana       0      0      0         5.5
```

Explanation

kabir sleeps the most, so he gets rank 1. rohan and sana tie at 0, so they share the average rank 5.5.

46. sort_index

sort_index() orders rows by their index labels.

Syntax

```
df.sort_index(ascending=True)
```

Example: Names in reverse alphabetical order

```
z_to_a = routine.sort_index(ascending=False)
print(z_to_a)
```

Output / Result

```
       sleep  study  phone  sleep_rank
name
sana       0      0      0         5.5
rohan      0      0      0         5.5
meera      5      2      7         4.0
kabir      8      6      2         1.0
diya       6      3      6         3.0
aarav      7      5      3         2.0
```

Explanation

The names in the index are sorted from Z to A.

47. set_index and reset_index

set_index() makes a column the index; reset_index() moves the index back into a normal column.

Syntax

```
df.set_index('col')
df.reset_index()
```

Example: Using ride_id as the index

```
by_id = rides.set_index('ride_id')
print(by_id[['city', 'fare']].head(3))

print(by_id.loc['RD24990', ['pickup_area', 'drop_area', 'fare']])

back = by_id.reset_index()
print(back[['ride_id', 'city']].head(3))
```

Output / Result

```
           city   fare
ride_id
RD25000  Mumbai  360.0
RD24999  Mumbai  509.0
RD24998    Pune  167.0
pickup_area    Lajpat Nagar
drop_area             Dwarka
fare                   108.0
Name: RD24990, dtype: object
   ride_id    city
0  RD25000  Mumbai
1  RD24999  Mumbai
2  RD24998    Pune
```

Explanation

With ride_id as the index, any ride can be fetched directly with loc['RD24990']. reset_index() moves ride_id back into a normal column.

48. unique and nunique

unique() lists distinct values; nunique() counts them.

Syntax

```
df['col'].unique()
df['col'].nunique()
```

Example: Ride types, drivers and restaurants

```
print(rides['ride_type'].unique())
print("Drivers    :", rides['driver'].nunique())
print("Restaurants:", food['restaurant'].nunique())
```

Output / Result

```
['Mini', 'Bike', 'Sedan', 'Auto', 'SUV']
Categories (5, str): ['Auto', 'Bike', 'Mini', 'SUV', 'Sedan']
Drivers    : 25
Restaurants: 14
```

Explanation

Because ride_type was converted to category in Topic 40, unique() also lists its categories. The rides were driven by 25 drivers, and the food came from 14 restaurants.

49. isnull, notnull and hasnans

notnull() is the opposite of isnull(); hasnans tells you if a column has any missing value.

Syntax

```
df['col'].notnull()
df['col'].hasnans
```

Example: Orders with and without a written review

```
has_review = food['review'].notnull()
print("Orders with a review:", has_review.sum())

print("review has NaN?      ", food['review'].hasnans)
print("order_value has NaN? ", food['order_value'].hasnans)
```

Output / Result

```
Orders with a review: 414
review has NaN?       True
order_value has NaN?  False
```

Explanation

notnull().sum() counts the filled cells: only 414 orders have a written review. hasnans is a quick True/False check for a whole column.

50. dropna

dropna() removes rows with missing values. subset limits the check to certain columns.

Syntax

```
df.dropna()
df.dropna(subset=['col'])
```

Example: Keeping only completed, rated rides

```
print("All rides:", rides.shape[0])

no_missing_at_all = rides.dropna()
print("dropna()                          :", no_missing_at_all.shape[0])

rated = rides.dropna(subset=['driver_rating'])
print("dropna(subset=['driver_rating']):", rated.shape[0])
```

Output / Result

```
All rides: 1000
dropna()                          : 0
dropna(subset=['driver_rating']): 795
```

Explanation

A plain dropna() removes EVERY ride - completed rides have no cancelled_by value and cancelled rides have no fare, so every row has at least one NaN. subset=['driver_rating'] keeps the 795 rides that were rated.

51. fillna

fillna() replaces missing values with a value you choose.

Syntax

```
df['col'].fillna(value)
```

Example: Filling missing coupons and ratings

```
coupons = food['coupon_code'].fillna('NO COUPON')
print(coupons.value_counts())

median_rating = food['customer_rating'].median()
print("Median rating:", median_rating)
ratings = food['customer_rating'].fillna(median_rating)
print("Missing ratings after fillna:", ratings.isnull().sum())
```

Output / Result

```
coupon_code
NO COUPON    1096
FEAST20       179
WELCOME50      96
BIRYANI30      90
WEEKEND15      42
Name: count, dtype: int64
Median rating: 4.0
Missing ratings after fillna: 0
```

Explanation

Missing coupons become 'NO COUPON' - most orders did not use one. Missing ratings are filled with the median rating (4.0), so no rating is left empty.

52. drop_duplicates

drop_duplicates() removes repeated rows. subset and keep control what is compared and kept.

Syntax

```
df.drop_duplicates()
df.drop_duplicates(subset=['col'], keep='first')
```

Example: Removing double-placed orders

```
print("Before:", food.shape)

food = food.drop_duplicates()
print("After :", food.shape)

latest_per_city = food.drop_duplicates(subset='city', keep='first')
print(latest_per_city[['order_date', 'city', 'restaurant',
                       'main_dish']])
```

Output / Result

```
Before: (1503, 23)
After : (1500, 23)
    order_date       city        restaurant        main_dish
0   2024-06-30     Mumbai  The Healthy Bowl     Protein Bowl
2   2024-06-30      Delhi  Paratha Paradise   Paneer Paratha
7   2024-06-30       Pune   Pav Bhaji Point        Pav Bhaji
12  2024-06-29  Bengaluru       Burger Barn    Cheese Burger
15  2024-06-29  Hyderabad  Biryani Junction  Chicken Biryani
```

Explanation

The 3 double-placed orders are removed (1,503 to 1,500 rows). Because the file is sorted newest-first, keeping the first row per city shows the latest order in every city.

53. drop

drop() removes columns by name or rows by index label.

Syntax

```
df.drop(columns=['c1', 'c2'])
df.drop(index=['label'])
```

Example: Removing columns and survey rows

```
print("Columns before:", rides.shape[1])

slim = rides.drop(columns=['rider', 'cancelled_by', 'country'])
print("Columns after :", slim.shape[1])

filled = routine.drop(index=['rohan', 'sana'])
print(filled)
```

Output / Result

```
Columns before: 20
Columns after : 17
       sleep  study  phone  sleep_rank
name
aarav      7      5      3         2.0
diya       6      3      6         3.0
kabir      8      6      2         1.0
meera      5      2      7         4.0
```

Explanation

Three columns are removed from rides, and the two people who have not filled the survey are removed from routine.

54. apply

apply() runs your own function on every value of a column (or on every row with axis=1).

Syntax

```
df['col'].apply(function)
```

Example: Labelling every delivery

```
def speed(minutes):
    if minutes <= 25:
        return 'Super Fast'
    elif minutes <= 40:
        return 'On Time'
    else:
        return 'Late'

food['delivery_speed'] = food['delivery_mins'].apply(speed)
print(food[['restaurant', 'delivery_mins', 'delivery_speed']].head())
print(food['delivery_speed'].value_counts())
```

Output / Result

```
         restaurant  delivery_mins delivery_speed
0  The Healthy Bowl             35        On Time
1  Biryani Junction             36        On Time
2  Paratha Paradise             49           Late
3  Biryani Junction             34        On Time
4  Paratha Paradise             25     Super Fast
delivery_speed
On Time       975
Late          296
Super Fast    229
Name: count, dtype: int64
```

Explanation

Each delivery time is passed to speed(), and the returned label is stored in the new delivery_speed column. 296 orders took more than 40 minutes.

55. isin

isin() checks each value against a list of values.

Syntax

```
df[df['col'].isin([v1, v2, v3])]
```

Example: Weekend food orders

```
weekend = food['day'].isin(['Saturday', 'Sunday'])
print(weekend.head())

print("Weekend avg delivery:",
      round(food[weekend]['delivery_mins'].mean(), 1), "mins")
print("Weekday avg delivery:",
      round(food[~weekend]['delivery_mins'].mean(), 1), "mins")
```

Output / Result

```
0    True
1    True
2    True
3    True
4    True
Name: day, dtype: bool
Weekend avg delivery: 36.4 mins
Weekday avg delivery: 32.6 mins
```

Explanation

isin() replaces writing (day == 'Saturday') | (day == 'Sunday'). Weekend deliveries take about 4 minutes longer - busy kitchens and busy roads!

56. corr

corr() measures how two numeric columns move together, from -1 to +1.

Syntax

```
df[['col1', 'col2']].corr()
```

Example: Does late delivery mean a lower rating?

```
numbers = food[['quantity', 'order_value', 'delivery_mins',
                'customer_rating']]
print(numbers.corr().round(2))
```

Output / Result

```
                 quantity  order_value  delivery_mins  customer_rating
quantity             1.00         0.64          -0.02             0.01
order_value          0.64         1.00           0.05            -0.05
delivery_mins       -0.02         0.05           1.00            -0.37
customer_rating      0.01        -0.05          -0.37             1.00
```

Explanation

delivery_mins and customer_rating have a negative correlation (-0.37): the longer the wait, the lower the rating. quantity and order_value are strongly positive - more items, bigger bill.

57. nlargest and nsmallest

nlargest(n, col) returns the n rows with the highest values; nsmallest(n, col) the lowest.

Syntax

```
df.nlargest(n, 'col')
df.nsmallest(n, 'col')
```

Example: Biggest food bills and fastest deliveries

```
biggest = food.nlargest(3, 'order_value')
print(biggest[['customer', 'restaurant', 'quantity', 'order_value']])

fastest = food.nsmallest(3, 'delivery_mins')
print(fastest[['restaurant', 'area', 'delivery_mins']])
```

Output / Result

```
      customer   restaurant  quantity  order_value
182  Ananya S.  Cake Corner         6         1930
448   Karan A.  Cake Corner         6         1930
864   Tanvi K.  Cake Corner         6         1930
          restaurant             area  delivery_mins
338   Tandoori Nights     Lajpat Nagar             12
639  The Healthy Bowl  Electronic City             12
926   Tandoori Nights       Karol Bagh             12
```

Explanation

The biggest bills are all 6-item Chocolate Truffle Cake orders (party time!). The fastest deliveries took just 12 minutes.

58. insert

insert() adds a new column at a chosen position.

Syntax

```
df.insert(position, 'col', values)
```

Example: Adding a city column at the front

```
routine.insert(0, 'city',
               ['Pune', 'Nashik', 'Mumbai', 'Pune', 'Nagpur', 'Pune'])
print(routine)
```

Output / Result

```
         city  sleep  study  phone  sleep_rank
name
aarav    Pune      7      5      3         2.0
diya   Nashik      6      3      6         3.0
kabir  Mumbai      8      6      2         1.0
meera    Pune      5      2      7         4.0
rohan  Nagpur      0      0      0         5.5
sana     Pune      0      0      0         5.5
```

Explanation

insert(0, ...) puts city as the first column. Normal assignment (df['city'] = ...) always adds a column at the end.

59. copy

copy() creates an independent copy - changing it never changes the original.

Syntax

```
new_df = df.copy()
```

Example: Why copy() matters

```
backup = routine.copy()
backup.loc['aarav', 'phone'] = 12

print("Copy     :", backup.loc['aarav', 'phone'])
print("Original :", routine.loc['aarav', 'phone'])
```

Output / Result

```
Copy     : 12
Original : 3
```

Explanation

Editing the copy leaves the original routine table untouched.

60. groupby

groupby() splits rows into groups and calculates something for every group.

Syntax

```
df.groupby('col')['value_col'].mean()
```

Example: Average fare per ride type and food orders per meal time

```
fare_by_type = rides.groupby('ride_type')['fare'].mean()
print(fare_by_type.round(0))

by_meal = food.groupby('meal_time')['order_value'].mean()
print(by_meal.round(0).sort_values(ascending=False))
```

Output / Result

```
ride_type
Auto     188.0
Bike     117.0
Mini     265.0
SUV      430.0
Sedan    335.0
Name: fare, dtype: float64
meal_time
Late Night    639.0
Dinner        582.0
Lunch         553.0
Snacks        434.0
Breakfast     375.0
Name: order_value, dtype: float64
```

Explanation

Bikes are the cheapest ride (about Rs. 117) and SUVs the costliest (Rs. 430). Late-night food orders have the biggest average bill, breakfast the smallest.

Teaching Note

groupby is covered in full detail in the next session - this is a first taste.

61. to_csv

to_csv() saves a DataFrame to a CSV file. index=False skips writing the row labels.

Syntax

```
df.to_csv('file.csv', index=False)
```

Example: Saving the late deliveries

```
late = food[food['delivery_speed'] == 'Late'][['order_id', 'restaurant',
                                               'delivery_mins']]
late.to_csv('late_deliveries.csv', index=False)

check = pd.read_csv('late_deliveries.csv')
print(check.shape)
print(check.head(3))
```

Output / Result

```
(296, 3)
  order_id        restaurant  delivery_mins
0  FD49998  Paratha Paradise             49
1  FD49995   Pav Bhaji Point             41
2  FD49991  Biryani Junction             47
```

Explanation

Reading the saved file back confirms that all 296 late deliveries were written with exactly three columns.